

Introduction to Numerical Methods -7-

11

Random numbers, fitting

Random numbers

Needed for:

- Cryptography
- Stochastic models, systems
- Statistical sampling (ensemble average)

Requirement:

- Cryptography
 - Uncorrelated
 - Truly random
 - Not too frequently
- Simulation
 - Uncorrelated
 - Fast!!!
 - Reproducible (for debugging)
 - Frequently

True random numbers

- Source: physical phenomenon
 - Shot noise (circuit)
 - Nuclear decay
 - Atmospheric noise
 - Lava lamps
- Advantages:
 - Truly unpredictable
- Disadvantages:
 - Slow ~ sec/min frequency
 - Expensive

Pseudo random numbers

- Generated by an algorithm
- Advantages:
 - Fast
 - Deterministic (debugging)
 - Cheap
 - Can be parallelized
 - Can be made reliable
- Disadvantages
 - Limited cycle
 - Can be a liability for crypto
 - Seed problem

Algorithm requirements

- performance
- Precision
- life cycle
- Randomness

Most used algorithm:

Multiplicative Congruential alg.

Let x_i be an integer number, the next

$$x_{i+1} = (a \cdot x_i + c) \bmod(m)$$

- Parameters: (a, c, m)
- If $m = 2^k$ then modulus can be replaced by $\&(m-1)$, which is a fast op on any processor (even ancient ones)
- Maximal life cycle: m
- Good parameters
 - Historical: $a = 7^5 = 16807$, $m = 2^{31} - 1$, $c = 0$
 - gcc built-in: $a = 1103515245$
 $m = 2^{31}$, $c = 12345$
- Bad parameters:
 - $a = 65539$, $m = 2^{31}$, $c = 0$

Permuted congruential algorithm

$$\text{count} = r_i \gg 61 \quad (\text{top 3 bits})$$

$$r_{i+1} = (a \cdot r_i + c) \bmod (m)$$

$$r = r \text{ XOR } (r \gg 22) \quad \text{cyclic shift}$$

$$r_{i+1} = r \gg (22 + \text{count})$$

The new number is the XOR-d of the MCA and its shifted itself
 numpy uses a variant of this.

Tests of randomness

- Birthday spacings (exponential)
- Montroy test (prop. of random sequences)
- Parking lot test
- Moments
- Correlation
- Fourier spectrum
- Fill of d dim lattice
- Random walks

Marsaglia effect: RANDU: generate $x_{15,2}$

the points lie on specific planes



Seed:

- True random source (good for crypto)
- Time : default option, bad practice
 - Debussing is hard
 - Runs launched at the same time are identical
- Manual : preferred option
 - Reproducible
 - Different runs can be made uncorrelated
 - Save the seed if rerun later

Distribution: Inverse transform sampling

Probability density function $f(x)$

Cumulative distribution is between $[0, 1]$ and not increasing

$$F(x) = \int_{-\infty}^x f(x') dx'$$

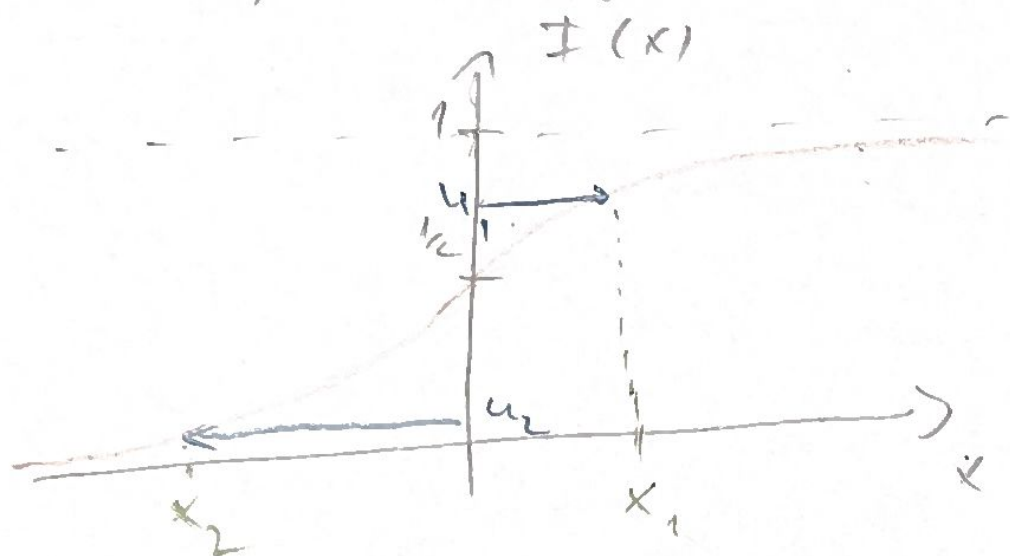
Find $T : [0, 1] \rightarrow \mathbb{R}$ such that $T(u) = X$

$$F_X(x) = \mathbb{P}(X \leq x) = \mathbb{P}(T(u) \leq x) = \mathbb{P}(u \leq T^{-1}(x)) = T^{-1}(x) \quad \text{for } x \in \mathbb{R}$$

$$\Rightarrow T(u) = F_X^{-1}(u) \quad u \in [0, 1]$$

$F^{-1}(u)$ is the inverse function of the cumulative probability distribution not always accessible analytically

e.g. $f(x) = \frac{1}{\pi} \frac{1}{1+x^2} \Rightarrow F(x) = \frac{1}{2} + \frac{1}{\pi} \arctan(x)$
 $x = F^{-1}(u) = \tan\left(\pi(u - \frac{1}{2})\right)$



Gaussian: Box-Müller method

Let (x, y) be a pair of independent normally distributed random numbers

$$f(x, y) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2} \cdot \frac{1}{\sqrt{2\pi}} e^{-y^2/2} = \frac{1}{2\pi} e^{-(x^2+y^2)/2}$$

$(x, y) \rightarrow (r, \varphi)$ $0 \leq \varphi \leq 2\pi$ uniform $\varphi = 2\pi U_1$

$x = r \cos \varphi$ $y = r \sin \varphi$

$$G(r) = P(r' \leq r) = \frac{1}{2\pi} \int_0^r \int_0^{2\pi} dr' d\varphi r' e^{-r'^2/2} = \int_0^r r' e^{-r'^2/2} dr' = 1 - e^{-r^2/2}$$

$$G(r) = 1 - e^{-r^2/2} = 1 - U_2$$

$$\Rightarrow r = \sqrt{-2 \log U_2}$$

U_2 uniform $(0, 1)$

We will have two

normal distributed numbers

$$Z_0 = \sqrt{-2 \log U_1} \cos(2\pi U_1), Z_1 = \sqrt{-2 \log U_1} \sin(2\pi U_1)$$